

Отчёт по лабораторной работе

Тема: Развертывание GPU-кластера Kubernetes

Студент: Умнов Сергей Алексеевич

1. Параметры VM и модель GPU

- Hostname: 111.88.153.248
- CPU: 8 vCPU
- CPU model: Intel Xeon Processor (IceLake)
- RAM: 31 GiB
- GPU: Tesla T4
- GPU memory: 15360 MiB

2. Версии окружения

- Ubuntu: Ubuntu 22.04.5 LTS
- Kubernetes: v1.34.8
- GPU Operator: v25.10.1
- NVIDIA driver: 580.105.08
- CUDA runtime: 12.4

3. Состояние ноды

Команда

```
kubectl get nodes -o wide
```

Вывод

sergejumnov148@epds73cmval2ppf6qhpm:~\$ kubectl get nodes -o wide										
NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME	
epds73cmval2ppf6qhpm	Ready	control-plane	140m	v1.34.8	10.129.0.25	<none>	Ubuntu 22.04.5 LTS	5.15.0-157-generic	containerd://2.2.1	

Узел находится в состоянии Ready . Кластер одноузловой, нода имеет роль control-plane . В качестве container runtime используется containerd 2.2.1 .

4. Состояние pod'ов

Команда

```
kubectl get pods -A
```

Вывод

sergejumnov148@epds73cmval2ppf6qhpm:~\$ kubectl get pods -A

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
gpu-lab	torch-benchmark-rhzw	0/1	Completed	0	57m
gpu-operator	gpu-feature-discovery-ddb4j	2/2	Running	0	131m
gpu-operator	gpu-operator-7569f8b499-x6zbz	1/1	Running	2 (119m ago)	132m
gpu-operator	gpu-operator-node-feature-discovery-gc-55ffc49ccc-8lbqm	1/1	Running	0	132m
gpu-operator	gpu-operator-node-feature-discovery-master-6b5787f695-zfgf8	1/1	Running	2 (120m ago)	132m
gpu-operator	gpu-operator-node-feature-discovery-worker-6l6g4	1/1	Running	0	132m
gpu-operator	nvidia-container-toolkit-daemonset-pgkw6	1/1	Running	0	131m
gpu-operator	nvidia-cuda-validator-4npxn	0/1	Completed	0	127m
gpu-operator	nvidia-dcgm-exporter-64hdt	1/1	Running	0	131m
gpu-operator	nvidia-device-plugin-daemonset-xwvt7	2/2	Running	0	93m
gpu-operator	nvidia-driver-daemonset-ppsd5	1/1	Running	0	131m
gpu-operator	nvidia-operator-validator-lwcf1	1/1	Running	0	131m
kube-flannel	kube-flannel-ds-mbgs7	1/1	Running	0	141m
kube-system	coredns-66bc5c9577-72lt2	1/1	Running	0	146m
kube-system	coredns-66bc5c9577-f824n	1/1	Running	0	146m
kube-system	etcd-epds73cmval2ppf6qhpm	1/1	Running	1	146m
kube-system	kube-apiserver-epds73cmval2ppf6qhpm	1/1	Running	1	146m
kube-system	kube-controller-manager-epds73cmval2ppf6qhpm	1/1	Running	11 (120m ago)	146m
kube-system	kube-proxy-lw4rf	1/1	Running	0	146m
kube-system	kube-scheduler-epds73cmval2ppf6qhpm	1/1	Running	11 (119m ago)	146m
monitoring	alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	0	102m
monitoring	monitoring-grafana-5fdd9f4674-j8j8f	3/3	Running	0	102m
monitoring	monitoring-kube-prometheus-operator-57bb844c8c-5nwk7	1/1	Running	0	102m
monitoring	monitoring-kube-state-metrics-7f7f8474d9-b4gn8	1/1	Running	0	102m
monitoring	monitoring-prometheus-node-exporter-zk8t2	1/1	Running	0	102m
monitoring	prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	0	102m

Все системные pod находятся в статусе Running или Completed . GPU Operator, Flannel и monitoring-stack успешно запущены.

5. Проверка GPU из pod

Команда

```
kubectl exec -n gpu-operator ds/nvidia-driver-daemonset -- nvidia-smi
```

Вывод

```
sergejumnov148@epds73cmval2ppf6qhpm:~$ kubectl exec -n gpu-operator ds/nvidia-driver-daemonset -- nvidia-smi
Sun May 31 17:14:41 2026

+-----+
| NVIDIA-SMI 580.105.08                Driver Version: 580.105.08          CUDA Version: 13.0               |
+-----+-----+-----+
| GPU   Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan   Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                               |                      | MIG M. |
+-----+-----+-----+
|  0    Tesla T4               On        | 00000000:8B:00:0  Off  |            0         |
| N/A    34C    P8              14W / 70W |  0MiB / 15360MiB |      0%      Default |
|                               |                      | N/A     |
+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
|       ID   ID              |              |           |         Usage |
+-----+-----+-----+
| No running processes found |
+-----+
```

6. Описание GPU-нагрузки

Для проверки работы GPU использовалась следующая PyTorch-нагрузка: повторяющееся умножение двух матриц на CUDA.

Основной фрагмент кода бенчмарка:

```

import os
import time
import statistics
import torch

if not torch.cuda.is_available():
    raise RuntimeError("CUDA is not available inside the pod")

device = torch.device("cuda:0")

print(f"GPU name      : {torch.cuda.get_device_name(0)}")
print(f"PyTorch version : {torch.__version__}")
print(f"CUDA version    : {torch.version.cuda}")

matrix_size = int(os.environ.get("MATRIX_SIZE", "8192"))
duration_sec = int(os.environ.get("DURATION_SEC", "300"))

A = torch.randn((matrix_size, matrix_size), device=device, dtype=torch.float32)
B = torch.randn((matrix_size, matrix_size), device=device, dtype=torch.float32)

for _ in range(10):
    C = torch.matmul(A, B)
    torch.cuda.synchronize()

torch.cuda.reset_peak_memory_stats()

step_times = []
steps = 0
start_total = time.perf_counter()

while time.perf_counter() - start_total < duration_sec:
    start = time.perf_counter()

    C = torch.matmul(A, B)
    torch.cuda.synchronize()

    end = time.perf_counter()

    step_times.append(end - start)
    steps += 1

peak_mem_gb = torch.cuda.max_memory_allocated() / 1024**3

print(f"steps          : {steps}")
print(f"total_time_sec  : {time.perf_counter() - start_total:.6f}")
print(f"avg_step_sec    : {statistics.mean(step_times):.6f}")
print(f"min_step_sec    : {min(step_times):.6f}")
print(f"max_step_sec    : {max(step_times):.6f}")
print(f"median_step_sec : {statistics.median(step_times):.6f}")
print(f"peak_cuda_mem_gb : {peak_mem_gb:.3f}")

```

Бенчмарк выполняется около 300 секунд, что позволяет увидеть долговременную нагрузку на GPU и получить значимые метрики.

7. Эксперимент 1: exclusive GPU

Результаты

В режиме exclusive была запущена одна Kubernetes Job, запрашивающая целую GPU через ресурс `nvidia.com/gpu: 1`. Внутри pod выполнялся PyTorch benchmark: повторяющееся умножение матриц размера 8192 x 8192 на CUDA в течение 300

секунд.

Результаты benchmark:

Метрика	Значение
Matrix size	8192 x 8192
Duration	300.08 sec
Steps	1047
avg_step_sec	0.286597
min_step_sec	0.253166
max_step_sec	0.303507
median_step_sec	0.292198
peak_cuda_mem_gb	1.008 GB

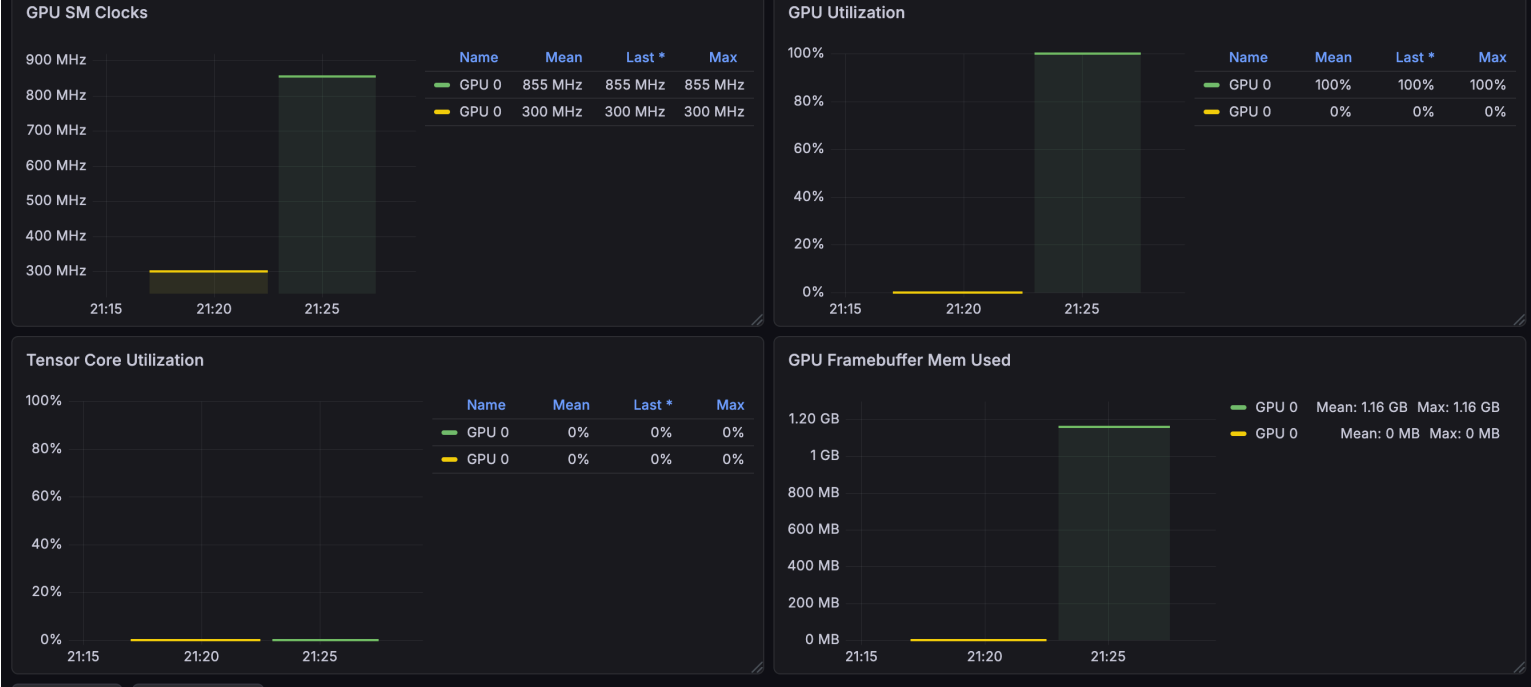
Метрики Grafana во время выполнения:

Метрика	Значение
GPU utilization max	100%
GPU memory used max	1.16 GB
GPU power usage max	69.8 W
GPU temperature max	46 °C

В режиме exclusive одна задача использовала всю физическую GPU, поэтому нагрузка стабильно достигала 100% utilization, а время шага оставалось достаточно стабильным.

Скрины





8. Эксперимент 2: time-slicing

Результаты

В режиме `time-slicing` был включён профиль `t4-timeslicing-4`, после чего Kubernetes стал видеть несколько логических `shared-ресурсов` `nvidia.com/gpu.shared`. Были запущены четыре параллельные Kubernetes Job, каждая выполняла ту же PyTorch-нагрузку: умножение матриц размера `8192 x 8192` на CUDA в течение 300 секунд.

Pod	Steps	total_time_sec	avg_step_sec	min_step_sec	max_step_sec	median_step_sec	peak_cuda_mem_gb
ts-1	218	300.746394	1.379561	1.018178	1.459004	1.379113	1.008
ts-2	218	300.806687	1.379838	1.034196	1.470798	1.378819	1.008
ts-3	218	300.793129	1.379776	1.027032	1.479405	1.379430	1.008
ts-4	273	300.240199	1.099772	0.302671	1.460675	1.371000	1.008

Метрики Grafana/DCGM во время выполнения:

Метрика	Значение
GPU utilization max	100%
GPU memory used max	4.63 GB
GPU power usage max	68.3 W
GPU temperature max	73 °C

В режиме `time-slicing` несколько `pod` одновременно использовали одну физическую GPU. По сравнению с `exclusive`, время шага отдельной задачи выросло: медианное время шага составило примерно `1.37 sec`. Это ожидаемо, так как одна GPU делится между несколькими `workload`.

У `pod ts-4` среднее время шага оказалось ниже, чем у остальных `pod`, поскольку в конце выполнения часть `pod` уже завершилась, и оставшийся `pod` получил больше вычислительного времени GPU. Поэтому для сравнения режимов использовалось `median_step_sec`, так как оно лучше отражает основную фазу параллельного выполнения.

Скрины



9. Эксперимент 3: MPS

Результаты

В режиме MPS был активирован профиль `t4-mps-4` из `ConfigMap device-plugin-config`. После переключения профиля в namespace `gpu-operator` появился pod `nvidia-device-plugin-mps-control-daemon`, что подтверждает запуск MPS control daemon:

```
nvidia-device-plugin-mps-control-daemon-zpvt4    2/2    Running
```

Далее были запущены четыре параллельные Kubernetes Job, каждая из которых выполняла тот же PyTorch benchmark: умножение матриц размера `8192 x 8192` на CUDA в течение 300 секунд.

Результаты выполнения benchmark:

Pod	Steps	total_time_sec	avg_step_sec	min_step_sec	max_step_sec	median_step_sec	peak_cuda_mem_gb
mps-1	171	300.554672	1.757620	1.565355	1.889372	1.789369	1.008

Pod	Steps	total_time_sec	avg_step_sec	min_step_sec	max_step_sec	median_step_sec	peak_cuda_mem_gb
mps-2	171	300.550657	1.757598	1.565393	1.889726	1.789362	1.008
mps-3	171	300.532250	1.757491	1.564895	1.889599	1.789601	1.008
mps-4	171	300.549915	1.757594	1.565243	1.889811	1.789416	1.008

Метрики Grafana/DCGM во время выполнения MPS workload:

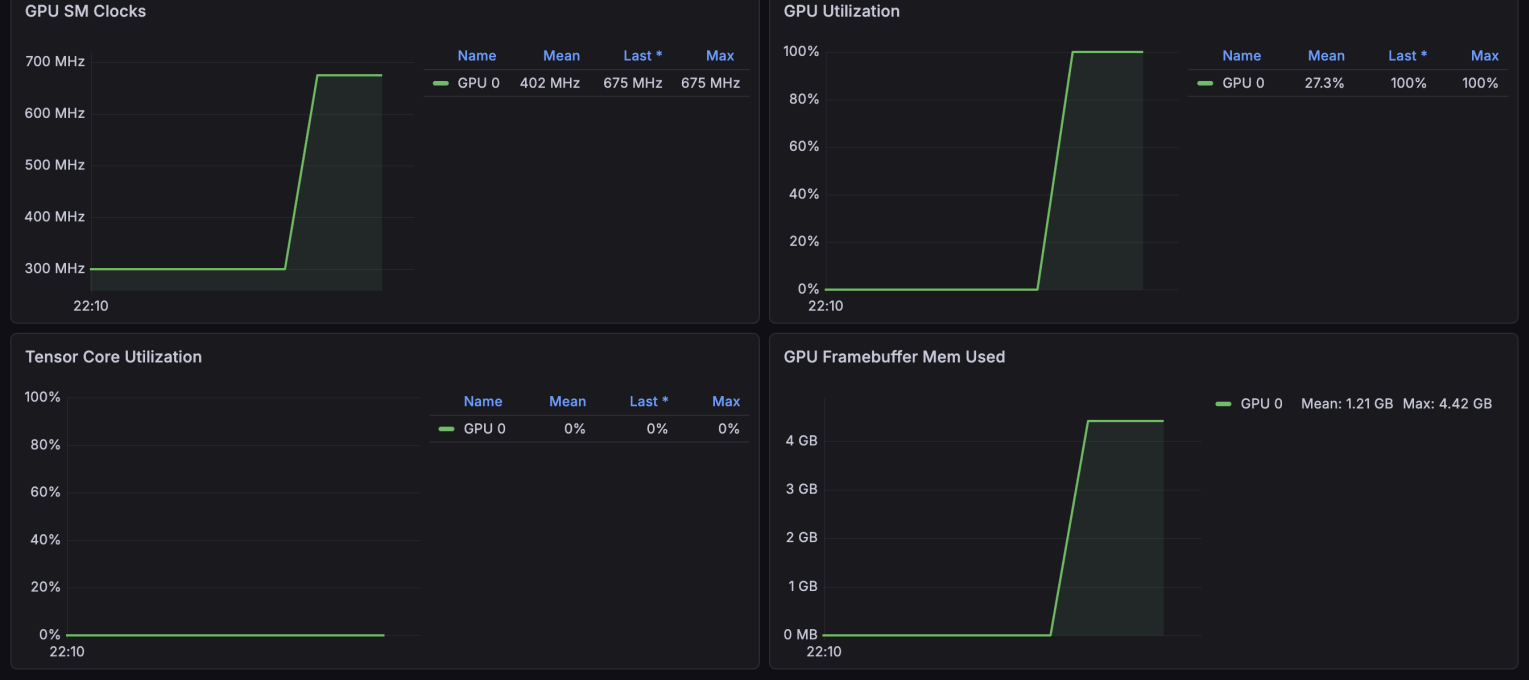
Метрика	Значение
GPU utilization max	100%
GPU memory used max	4.42 GB
GPU power usage max	69.8 W
GPU temperature max	72 °C

В режиме MPS все четыре pod показали практически одинаковые результаты: каждая задача выполнила 171 шаг, а медианное время шага составило около 1.789 sec . Это говорит о достаточно равномерном распределении GPU-ресурса между параллельными workload.

Таким образом, MPS-режим в данном окружении был успешно запущен. Он позволил одновременно выполнять четыре GPU workload на одной физической Tesla T4. По сравнению с time-slicing , MPS показал более равномерное поведение между pod, однако отдельная задача выполнялась медленнее: медианное время шага выросло примерно до 1.789 sec .

Скрины





10. Сравнение результатов

Так как benchmark запускался с фиксированной длительностью 300 sec, фактическое время выполнения job во всех режимах близко к 300 секундам. Поэтому для сравнения производительности дополнительно учитывалось среднее/медианное время одного шага вычисления.

Режим	Число pod	Kubernetes resource	Среднее время job	Пик GPU util	Пик FB used	Комментарий
Exclusive	1	nvidia.com/gpu	300.08 sec , avg step 0.2866 sec	100%	1.16 GB	Одна job получает всю физическую GPU, поэтому показывает лучшее время шага
Time-slicing	4	nvidia.com/gpu.shared	300.65 sec , median step ~1.37 sec	100%	4.63 GB	Четыре pod делят одну GPU; отдельная задача становится медленнее, но можно запускать несколько workload параллельно
MPS	4	nvidia.com/gpu.shared	300.55 sec , median step ~1.79 sec	100%	4.42 GB	Четыре pod работали одновременно через MPS; распределение между pod получилось более равномерным

11. Ответы на вопросы

Почему в exclusive одна job обычно выполняется быстрее?

В режиме exclusive одна job получает доступ ко всей физической GPU.. Поэтому latency одного шага минимальна: в эксперименте среднее время шага составило около 0.2866 sec .

Почему в time-slicing можно запустить несколько pod на одной GPU?

В режиме time-slicing NVIDIA device plugin публикует одну физическую GPU как несколько логических shared-ресурсов, например nvidia.com/gpu.shared: 4 . Кубер же видит эти ресурсы как доступные для планирования и может назначить нескольким pod по одному shared GPU-ресурсу.

Какие риски появляются при разделении одной GPU между несколькими pod?

При разделении одной GPU между несколькими pod появляются риски конкуренции за ресурсы. Один pod может начать активно использовать GPU-память или вычисления и повлиять на latency других pod. Также снижается предсказуемость времени выполнения: задачи могут работать медленнее и менее стабильно, чем в exclusive-режиме. Кроме того, shared GPU не даёт такой же строгой изоляции, как отдельная физическая GPU.

Чем MPS концептуально отличается от обычного time-slicing?

Time-slicing в основном разделяет GPU между процессами по времени: разные workload получают возможность использовать GPU по очереди. MPS работает иначе: NVIDIA Multi-Process Service позволяет нескольким CUDA-процессам совместно использовать GPU более кооперативно, через MPS control daemon. В результате MPS может уменьшать накладные расходы переключения между процессами и более равномерно распределять выполнение CUDA workload. В эксперименте все четыре MPS pod показали почти одинаковые значения времени шага: около 1.79 sec .

Какой режим вы бы выбрали для интерактивного JupyterLab, а какой для batch-задачи?

Для интерактивного JupyterLab я бы выбрал time-slicing или MPS , потому что несколько пользователей или notebook-сессий могут одновременно использовать одну GPU. Это удобно, когда задачи не нагружают GPU постоянно на 100%.

Для тяжёлой batch-задачи, например обучения модели или длительного benchmark, я бы выбрал exclusive , потому что он даёт одной job всю GPU, минимальное время шага и наиболее предсказуемую производительность.

12. Приложенные YAML-файлы

К отчёту приложены YAML-файлы:

- crd-timeslicing.yaml — конфигурация NVIDIA device plugin для режима Time-Slicing;
- crd-mps.yaml — конфигурация NVIDIA device plugin для режима MPS;
- torch-benchmark-shared.yaml — ML Task для shared GPU-режимов Time-Slicing и MPS;
- torch-benchmark-exclusive.yaml — ML Task для exclusive-режима.